LA HERENCIA EN LA PROGRAMACIÓN ORIENTADA A OBJETOS

La **herencia** es uno de los principios fundamentales de la Programación Orientada a Objetos (POO). Permite que una clase reutilice atributos y métodos de otra, fomentando la **reutilización del código**, la **extensibilidad** y la **modularidad** (Deitel & Deitel, 2016). Con este mecanismo es posible modelar jerarquías que representan relaciones del mundo real, como la especialización de categorías.

Definición y clasificación

La **herencia** es el proceso mediante el cual una clase (**subclase** o **clase derivada**) adquiere las propiedades y comportamientos de otra (**superclase** o **clase base**).

Tipos comunes de herencia:

- **Herencia simple**: una clase hereda de una única superclase.
- **Herencia múltiple**: una clase hereda de más de una superclase (en Java no está permitida con clases, pero sí mediante interfaces).
- **Herencia multinivel**: una clase hereda de otra, que a su vez hereda de otra.
- **Herencia jerárquica**: varias subclases heredan de la misma superclase.

```java
class Persona {  1 usage  1 inheritor
    String nombre;  1 usage
    void saludar() {  no usages
        System.out.println("Hola, soy " + nombre);
    }
}

class Estudiante extends Persona {  no usages
    int codigo;  no usages
    void estudiar() {  no usages
        System.out.println("Estudiando...");
    }
}
```

Clases abstractas

Una **clase abstracta** no puede ser instanciada y sirve como plantilla para que otras clases implementen sus métodos. Puede contener métodos implementados y **métodos abstractos** (sin cuerpo).

```java
abstract class Figura {  1 usage  1 inheritor
    abstract double calcularArea(); // Método abstracto  no usages  1 implementation
}

class Circulo extends Figura {  no usages
    double radio;  3 usages

    Circulo(double r) {  no usages
        radio = r;
    }

    @Override  no usages
    double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```

Constructores

Los **constructores** no se heredan, pero las subclases pueden invocar al constructor de la superclase con la palabra clave super.

```java
class Persona {  1 usage  1 inheritor
    String nombre;  1 usage

    Persona(String nombre) {  1 usage
        this.nombre = nombre;
    }
}

class Profesor extends Persona {  no usages
    String materia;  1 usage

    Profesor(String nombre, String materia) {  no usages
        super(nombre); // Llama al constructor de Persona
        this.materia = materia;
    }
}
```

Redefinición de métodos

La **redefinición** o **sobreescritura** permite a una subclase modificar el comportamiento de un método heredado. En Java se utiliza la anotación @Override.

```java
class Persona {  1 usage   1 inheritor
    void trabajar() {  no usages   1 override
        System.out.println("La persona está trabajando.");
    }
}

class Ingeniero extends Persona {  no usages
    @Override  no usages
    void trabajar() {
        System.out.println("El ingeniero está desarrollando software.");
    }
}
```

Restringir la herencia

En algunos casos no se desea que una clase pueda ser heredada o que un método pueda ser redefinido.

- En Java se utiliza la palabra clave final.

Ejemplo en Java:

```java
final class Utilidades {  no usages
    static void imprimirMensaje() {  no usages
        System.out.println("Mensaje desde una clase final.");
    }
}
```

Visibilidad protegida

El modificador de acceso **protected** permite que los miembros de una clase sean accesibles desde:

1. La misma clase.
2. Subclases, incluso si están en paquetes distintos.
3. Clases dentro del mismo paquete.

Ejemplo en Java:

```java
class Persona {  1 usage  1 inheritor
    protected String documento; // accesible en subclases  1 usage
}

class Estudiante extends Persona {  no usages
    void mostrarDocumento() {  no usages
        System.out.println("Documento: " + documento);
    }
}
```

Referencias

Barnes, D. J. (2007). *Programación Orientada a Objetos con Java: una introducción práctica usando BlueJ*. Pearson Educación.

Deitel, H. M., & Deitel, P. J. (2016). *Cómo programar en Java* (10.ª ed.). Pearson.

Joyanes, L. (1996). *Programación orientada a objetos*. McGraw-Hill.

López, R. L. (2006). *Metodología de la programación orientada a objetos*. Alfaomega.

Wu, C. (2008). *Programación en Java: introducción a la programación orientada a objetos*. McGraw-Hill.